Response to Reviewer (Revision Round 5)

Dear Editor and Reviewer,

Thank you for the exceptionally careful fifth reading of our manuscript, *A Reproducible Benchmark of Relational Database Access-Layer Performance in Express.js: A Configuration-Specific Comparison on PostgreSQL and MySQL*, and for judging the remaining issues "major but potentially correctable" with "a credible publication path in IST." We have taken every point seriously and made the strongest possible response to the one you ranked potentially fatal.

Below we answer point by point, following the structure of the review (§6.1–§6.10, §7 minors, §8 statistics, §9 artifact, §11 presentation, and the eight questions of §13). Section and line references are to the line-numbered `review`-class build of the revised submission; we quote the revised manuscript verbatim so every claimed change is verifiable.

—

The governing change this round: version currency (your Priority-1 concern)

You were right, and the correction has become the paper's spine. The earlier headline was measured on **Prisma 5.22**, whose in-process Rust query engine produced the ~498% / five-application-core CPU anomaly, at a time when **Prisma 7 (the Rust-free rewrite) was the contemporary stable release**. Our freeze rule justified only "install on the measurement date," which does not justify a superseded major version.

We did not patch the prose around this; we **re-froze every library to the latest stable release compatible with the harness at the freeze date (2026-07-15) and re-ran the entire 90-cell primary matrix and every secondary experiment.** The result vindicates both your concern and the paper's own thesis. On Prisma 7.8 (Rust-free) the five-core anomaly is gone — the maximum application-side CPU across all 90 cells is now 110% — and Prisma falls to mid-to-low throughput (last on the MySQL insert, near the bottom of aggregation). The old "Prisma ties native at a CPU premium" finding is **retired**, and we reframe the old→new contrast as the paper's central demonstrated claim: rankings are perishable; the reusable instrument, design, and pitfalls checklist are the durable contribution.

The pre-specified freeze rule is now stated in the Study Design (§Experimental setup, lines 321–332):

> "Each library is pinned to the *latest stable release compatible with the harness > adapter contract* as of the freeze date (2026-07-15), recorded from the committed > lockfile and an automated environment capture (Supplement Table S11); a library is > pinned to an earlier release only where the latest stable breaks the contract or > the byte-equivalence cross-check, and any such deviation is documented per layer. > Only Sequelize invokes this exception: its version-7 line is a prerelease, so the > current 6.x stable line is used. [...] Because access-layer performance is > version-sensitive, this pins a dated snapshot rather than a permanent ranking, > exactly one headline of which a superseded pin produced and a re-freeze retired."

The retirement is stated head-on in the Discussion (lines 4–24):

> "An earlier campaign [...] found the ORM Prisma (then version 5.22, built on an > in-process Rust query engine) tying the native driver on the deep/nested fetch > while drawing about five application cores [...]. Re-running the *identical* > harness, workload, and host against the latest stable releases (Prisma 7.8, now a > Rust-free TypeScript client executing through a JavaScript driver adapter) retired > that finding outright. No layer now draws more than about

one application core > (the maximum application-side CPU across all 90 cells is 110%), and Prisma has > fallen to mid-to-low throughput across the matrix [...]. Nothing changed but one > dependency's internals across a single major version. This is the paper's central > claim made concrete: access-layer performance is highly sensitive to > implementation and version, so any specific ranking is perishable, and the durable > contributions are the reusable harness, the benchmark design, and the pitfalls > checklist."

The pinned-version table (Supplement Table S11) now records `@prisma/client` 7.8.0, `drizzle-orm` 0.45.2, `typeorm` 1.1.0, `@mikro-orm/core` 7.1.6, `express` 5.2.1, `autocannon` 8.0.0, `mysql2` 3.23.0, and `sequelize` 6.37.8 (documented exception).

This re-run makes Questions 1 and 2 largely moot; we answer them explicitly in §13.

Major concerns (§6)

§6.1 — Saturated p99 is a saturation outcome; matched utilization elevated to co-primary

We agree, and the paper now names the saturated tail as an overload/queueing measurement and elevates the matched-utilization result to a co-primary latency construct in the *main text*. The Tail-latency subsection (Results, lines 165–171) now opens:

> "so the tail includes connection-acquisition queueing — the tail a deployment sees > at saturation, not intrinsic per-request latency, which the open-loop experiment > below reports below saturation."

and closes with the utilization result (lines 193–203):

> "Offering each layer 50/70/85/95% of *its own* saturating throughput [...] the tails > converge: at 50% utilization every layer holds p99 near 2–5 ms on both engines > regardless of its saturated rank [...]. At matched utilization the large saturated > gap (`pg` 20 ms versus MikroORM 116 ms) therefore largely dissolves; it is a > capacity-and-queueing effect, not a difference in intrinsic tail latency."

The abstract now states the same demotion: "Large saturated-p99 gaps are a capacity effect that matched utilization largely closes." The interpretation is thus propagated to the abstract, and via the Discussion "Queueing under load" paragraph and the Threats "internal validity" paragraph, closing the propagation gap you identified as recurring.

§6.2 — The 50-connection point now has an external rationale: three named estimands

We stop privileging the equal-demand point implicitly. The Study Design (lines 22–33) now names three operating conditions and maps each to its experiment:

> "*Equal external demand* is the saturated closed-loop matrix, every layer > receiving the identical request stream at a fixed 50-connection load. *Equal > utilization* is the coordinated-omission-corrected open-loop sweep, each layer > offered a fixed fraction (50/70/85/95%) of its *own* saturating throughput. An > *equal compute budget* is the equal-CPU control, the server confined to a fixed > number of cores [...]. These answer different questions and need not agree — the > saturated condition measures capacity under overload, matched utilization the > near-intrinsic per-request latency, the equal budget the compute cost of the same > work — so we report all three and say which each result speaks to."

RQ1 (Introduction, lines 79–83) now explicitly asks how the difference "depend[s] on the operating condition: equal external demand, equal utilization, or an equal compute budget."

§6.3 — "idiomatic" construct validity; "abstraction cost" removed

Two changes. First, the adapter-selection rule is now pre-specified and auditable (Study Design, lines 123–128):

> "The idiomatic API was fixed by a rule decided *before* measurement — the > eager-loading API each layer's official documentation presents first for the > pinned version — so the treatment is documented, not editorial; Supplement Table > S16 and `METHODOLOGY.md` record, per layer, that API with its **documentation page > and version**, its round-trip count, the documented alternative we did *not* use, > and that query logging, hooks, and validation are off."

The no-maintainer-review disclosure is retained (lines 48–51): "no evaluated library's maintainers were involved or invited to inspect the adapters, which does not remove the consequential choices any benchmark author makes [...]; we disclose these and release the harness for inspection."

Second, we replaced the value-laden "abstraction cost" throughout with the neutral *observed implementation-and-strategy difference* (a `grep` for "abstraction" now returns zero interpretive uses); e.g. the Discussion (line 27) reads "the idiomatic implementation-and-strategy difference is specific to the access *pattern*."

§6.4 — "equal configured connection limit"; same-SQL as a compound bounding contrast

The pool estimand is renamed and its sufficiency shown (Threats, external validity, lines 64–69):

> "The fixed pool makes this an *equal configured connection limit* comparison, not > a per-layer-tuned one: it isolates the access layer from pool tuning but does not > show each layer's throughput under its own best pool. A per-layer pool-size > frontier on both engines (Supplement Tables S7 and S24) addresses this directly: > the ordering is preserved from a pool of 1 to 50, and each layer's throughput > saturates well below 50, so the fixed-pool choice does not drive the ranking."

The same-SQL control is kept, but framed strictly as a compound bounding contrast that isolates no single factor (Results, lines 74–77):

> "The contrast is composite — it changes query strategy, query count, > prepared-statement behavior, the raw versus ORM API, and result marshalling > together — so the residual difference is bounded but no single factor is > isolated."

The residual-overreach phrasings you flagged ("abstraction is cheap when a layer merely forwards"; "recovers most of the deep fetch's cost") have been removed.

§6.5 — Single-host scope; the node-cluster result; RQ1 "overhead" → "performance difference"

RQ1 no longer presupposes a direction. Its results heading (line 26) now reads "**RQ1: performance difference relative to the native baseline**," and the RQ itself (Introduction, line 79) asks about the "throughput and response-time-p99 *difference* [...] relative to the native-driver baseline."

The single-host scope is declared (Study Design, lines 58–61: "which factors are held constant, which vary, and which are uncontrolled on this single virtualized host — the reason we report results for this configuration rather than as general library performance"). The feasible test of your specific worry — that a low single-process layer might merely be leaving cores idle — is the node-cluster experiment (Results, lines 270–276):

> "under a shared-port `node` cluster (Supplement Table S23) Prisma rises from 1,117 > to 2,275 to 4,449 req/s at 1, 2, and 4 workers on PostgreSQL, reaching native-like > aggregate throughput at four workers [...]. Prisma's low single-process throughput > is thus recoverable by running more processes, at roughly four times the cores."

§6.6 — Writes transfer *moderately*; statistics broadened (with the honest partial)

The write ranking is no longer said to "not transfer." It now transfers *moderately* (Discussion, lines 74–76):

> "The write ranking transfers only moderately across engines (Spearman $\{}rho=0.68$, > against 0.86–0.89 for the reads), so a single-engine write comparison invites the > wrong generalization either way [...]."

On the statistics we made three additions and disclose one partial. (i) Every primary throughput cell now carries a within-campaign 95% bootstrap CI in the tables (`deep_fetch`, `write`; e.g. `pg` deep fetch 3,687 [3,602–3,737], `mysql2` insert 1,753 [1,717–1,763]). (ii) The paired adjacent-rank significance table (Table `tab:significance`) reports, per adjacent pair, the geometric-mean paired ratio with a paired bootstrap CI, dominance, and permutation *p*. (iii) The rank table (`tab:ranks`) backs the correlations with per-layer PG-vs-MySQL ranks ($\{}rho=0.86$ deep fetch, 0.68 insert; Prisma rank 4→7).

We are explicit that the **effect-size-table extension is partial**: the full adjacent-pair permutation table is provided for the deep/nested fetch (the pattern that carries the pairwise claims), while writes and the cross-engine contrast are reported through the within-campaign CIs and the rank table descriptively; and the p99 columns remain point medians rather than carrying their own CIs. We preferred to disclose this scope than to over-generate tables that the RQ claims do not rest on.

§6.7 — Novelty search made auditable

The literature search is now reproducible (Related Work, lines 6–21):

> "We located it with a structured scoping search (not a systematic review) of the > ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (July 2026, over > titles, abstracts, and keywords), crossing an access-layer set [...] with context > [...], engine [...], and measurement [...] sets, restricted to 2006–July 2026 and > followed by backward and forward citation chaining from the closest hits. We > retained empirical comparisons of relational access layers or engines and excluded > non-empirical, non-relational, and single-library studies. The exact strings, > per-source dates, and screening decisions are archived with the replication > package (`notes/related-work-search.md`); because this is a scoping rather than > exhaustive search, we scope every resulting gap claim to the sources searched."

"we found none" is now everywhere softened to "in the sources searched."

Statistics (§8)

- **CI relabelling.** The bare "95% CI" is now labelled a *within-campaign* repeated-run

interval in the table captions ("median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days").

- **Blocked-permutation spec.** The interaction test is specified compactly (Study

Design, lines 291–296): "for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference $D_{\{}ell,i}$ and, under the null of no interaction, permute the layer labels of $\{}{D_{\{}ell,i}\{}}$ *within* each replicate i (20,000 permutations); the statistic is the between-layer F of a randomized-block ANOVA on D with replicate as the block." The within-replicate permutation is the exchangeability statement.

- **TOST sharpened.** The equivalence margin is now stated as author-selected, not a

general threshold (lines 302–312): "The $\{}pm5\{}%$ equivalence margin is an author-selected smallest effect of practical interest for *this* study, not a general engineering threshold [. . .]. We chose it before the analysis but did not preregister it."

- **Post-selection stated.** The adjacent-rank comparisons are treated descriptively:

"Because the seven adjacent pairs follow the *observed* ranking rather than a preregistered one, we treat their significances descriptively (post-hoc adjacent-rank comparisons)."

- **p99 sample size / histogram resolution.** See PENDING CHANGE 1 above and Q6 below.

Artifact (§9)

- **Prisma 7 driver adapters disclosed.** Supplement Table S11's note and the Threats

section (lines 80–85) disclose the new architecture and its one asymmetry: "Prisma 7 is Rust-free and connects through an official JS driver adapter (`@prisma/adapter-pg` for PostgreSQL, `@prisma/adapter-mariadb` with the `mariadb` 3.5.3 driver for MySQL, as Prisma ships no `mysql2` adapter)" — "an asymmetry with the other MySQL layers."

- **Versions table refreshed** to the re-frozen pins (S11, above).
- **Clean-room reproduction.** From the archived Zenodo tarball (release v1.4.1, DOI

10.5281/zenodo.21373621) — not the development checkout — a one-command setup regenerates every main-text and supplement table from the raw per-cell records; the harness is "released so the full matrix can be re-run with one command against a containerized or local PostgreSQL and MySQL" (Study Design, replication package). This confirms the archived package regenerates the tables (Question 7).

Presentation (§11) and minors (§7)

- **Length.** Counting references and floats per the IST guide (each float 200 words,

references included), the main text is now **14,917 words**, under the 15,000-word limit (`word-count.md`); 27 tables and one figure are in the numbered online supplement.

- **Structure.** The paper is organized around three findings — workload-dependent

differences (RQ1/RQ3), read-vs-write engine transfer (RQ2), and capacity-vs-utilization (Tail latency) — with defensive sensitivity prose moved to the supplement.

- **byte-identical* data *vs physical state.** Corrected (Study Design, lines 90–93):

"every engine receives the same seed *data payload* (byte-identical row values); the on-disk physical representation, index layout, and collation naturally differ between engines."

- **RAM working-set caveat softened** (lines 94–95): "the measurements *emphasize*

the access layer's per-request instruction, protocol, and mapping cost with disk-read latency largely removed; engine-side query execution, buffer management, locking, and logging remain part of every measurement."

- **"1.5× ... small" removed** from the abstract, which now reports the same-SQL spread numerically ("narrowing to 1.68× and 2.0×") without the editorializing adjective.
- The Knex n=24 exclusion is noted as prespecified (Study Design, lines 209–213).

New pitfall added by this very re-run: MikroORM 7 removed `persistAndFlush`

The re-run surfaced a fresh, current instance of the "correctness before timing" pitfall, which the checklist now includes. Results (Measurement stability, lines 213–217):

> "on the current versions MikroORM 7 removed `persistAndFlush`, so every MikroORM > *write* threw an HTTP 500 — which returns faster than a real insert, so the broken > cells briefly showed *inflated* throughput and spuriously led the MySQL insert > until the non-2xx counts flagged them."

Discussion (lines 96–102) generalizes it: "Timing is meaningless until every response is a real 2xx, so the error counter is the write path's correctness gate as byte-equality is the read path's." The supplement checklist item 1 ("Correctness before timing") covers it.

—

The eight questions (§13)

Q1 — Why Prisma 5.22 in July 2026? Under the old install-on-date rule it was the copy installed then; the revision re-freezes to the latest stable (Prisma 7.8) and documents a per-layer freeze rule, so the concern is moot and, more importantly, turned the paper's thesis into a demonstrated result (Study Design lines 321–332; Discussion lines 4–24).

Q2 — Exact version rule, fixed before results? Yes: the Stage-0 latest-stable rule quoted above, applied uniformly, with deviations documented (Sequelize 7 is a prerelease, so the 6.x stable line is used; Prisma's MySQL adapter asymmetry is disclosed). No result influenced a version choice.

Q3 — Why 50 connections as the primary point? It is the *equal-external-demand* estimand and is no longer privileged: equal utilization and an equal compute budget are now co-primary operating conditions, each mapped to its experiment (Study Design lines 22–33).

Q4 — Would the conclusions change if matched utilization were primary? They change in exactly the way you anticipated, and we now foreground it: the large saturated gaps largely reflect capacity and queueing, so at matched utilization the gap "largely dissolves; it is a capacity-and-queueing effect, not a difference in intrinsic tail latency" (Results lines 199–201). The matched-utilization result is now co-primary in the main text.

Q5 — Were the adapters reviewed by the libraries' maintainers? No, and this is disclosed as a limitation (Study Design lines 48–51); the alt-loading variants (Supplement Table S18) and the byte-equivalence cross-check bound how far idiomatic representativeness could be off.

Q6 — How many requests contribute to each run-level p99 in the slowest cells, and at what histogram resolution? `autocannon` records a per-run latency histogram at 1

ms resolution; a 12 s measured run of the slowest deep-fetch cells (≈480–516 req/s) contains ≈5,760–6,190 completed requests, so each run-level p99 is estimated from roughly 58–62 requests in the upper 1% of a single run, and the reported p99 is the median of 25 such run-level values. **This sentence is not yet in the manuscript — see PENDING CHANGE 1**; we will add it to the Measurement procedure after verifying the counts against the archived histograms.

Q7 — Does the Zenodo archive regenerate every table clean-room? Yes. A clean-room reproduction from the archived v1.4.1 tarball (not the dev checkout) regenerates every main-text and supplement table from the raw per-cell records via the committed one-command pipeline; the table-to-record manifest (`MANIFEST.md`) traces each cell to its source.

Q8 — Which claims survive anonymizing the products to A–I? The *durable* claims survive the relabelling: (a) the difference is workload-dependent, concentrating in relation-materializing patterns and small where a layer forwards SQL; (b) read rankings transfer across engines while aggregation and writes transfer only moderately (an asymmetry visible only in the dual-engine design); and (c) large saturated-p99 gaps are a capacity-and-utilization effect, not intrinsic tail latency. What does *not* survive anonymization is the specific leaderboard — precisely the perishability point the retired Prisma result now demonstrates. The contribution is the instrument and the design that let any current leaderboard be re-established, not the leaderboard itself.

—

Summary of Changes

For the editor's convenience:

- **Full re-freeze and re-run.** Every library pinned to the latest stable release

at the 2026-07-15 freeze date; the entire 90-cell matrix and all secondary experiments re-measured (Study Design §Experimental setup; Supplement Table S11).

- **Prisma five-core headline retired** and reframed as the paper's central

demonstrated claim that rankings are perishable (Discussion; abstract; Introduction contributions; Conclusion).

- **Three named operating conditions** (equal demand / equal utilization / equal

compute budget), each mapped to an experiment; matched utilization elevated to co-primary in the main text (Study Design; Results Tail latency; abstract).

- **RQ1 reworded** from "overhead" to "performance difference relative to native"

(Introduction; Results heading).

- **Pre-specified idiomatic-selection protocol** and no-maintainer disclosure (Study

Design; Supplement Table S16, `METHODOLOGY.md`).

- **"equal configured connection limit"** estimand + pool frontier (Threats);

same-SQL kept as a compound bounding contrast (Results; Study Design).

- **Writes transfer moderately** ($\{}rho=0.68$) rather than "does not transfer"

(Discussion; `tab:ranks`).

- **Statistics:** within-campaign CIs on every primary throughput cell; paired

adjacent-rank significance table; per-layer rank table; blocked-permutation spec; TOST margin stated as author-selected; effect-size extension disclosed as partial.

- **Auditable novelty search** (Related Work; `notes/related-work-search.md`).
- **New "correctness before timing" pitfall** (MikroORM 7 `persistAndFlush`)

(Results; Discussion; Supplement checklist).

- **Artifact:** Prisma 7 driver adapters disclosed; versions table refreshed;

clean-room reproduction from the archived tarball confirmed.

- **Presentation:** 14,917 words (under 15,000); byte-identical *data* vs physical

state; RAM caveat softened; "1.5× … small" removed.

We are grateful that your Priority-1 concern pushed us to the re-run; it converted the paper's thesis from an assertion into a measured result and, we believe, leaves the manuscript claiming exactly what a single-host, current-version benchmark can identify. We look forward to your assessment.

Sincerely,

Mateusz Miotk Faculty of Mathematics, Physics and Informatics, University of Gdańsk